

Union-Find

A study guide using the CEO & acquisitions analogy

What this guide covers: the union-find data structure with union-by-rank and path compression, explained through a story about five people at startups acquiring each other. By the end you'll be able to write the algorithm from memory and visualize what happens at every step.

The problem we're solving

You have a bunch of items. Over time, you learn that some pairs are connected (friends, members of the same group, points in the same cluster). You want to answer one question fast:

Are these two items in the same group?

Union-find solves this efficiently. The clever bit is that everything is built from just two operations — `find` and `union` — backed by a single array of pointers.

The core idea: trees of pointers

We represent each group as a tree. Every item points to a 'parent.' The **root** of the tree (an item that points to itself) is the group's identity. Two items are in the same group if and only if they share the same root.

We store this in a single dictionary called `parent`. Initially, everyone points to themselves — every item is its own group of one.

```
parent = {
    'A': 'A',
    'B': 'B',
    'C': 'C',
    'D': 'D',
}

# Read parent['B'] = 'A' as: "B's boss is A."
```

The analogy: chain of command

Think of each tree as a company's chain of command:

In code	In the analogy
each item	an employee or founder
parent[x] = y	x's boss is y
parent[x] = x	x is a CEO (their own boss)
the root of x's tree	x's top-level CEO
find(x)	walk up x's chain of command to find the CEO
union(a, b)	two companies merge — one CEO becomes the other's boss
rank[ceo]	roughly how tall the CEO's company chart is

The mental model: 'Who do you ultimately work for?' = walk up the boss chain until you hit someone who's their own boss. That's your CEO. Two people are in the same company if they walk up to the same CEO.

The story: five people, one big acquisition

Cast of characters

Kyle	junior employee
Eddie	startup founder
Nambi	junior employee
Arye	startup founder
Rock	CEO of a large established company called Rockwell

The sequence of events

1. Eddie hires Kyle
 2. Arye hires Nambi
 3. Arye acquires Eddie's company
 4. Rockwell (already big) acquires Arye's company
 5. Someone asks: who does Kyle ultimately work for?
-

STEP 0 Initial state — everyone is their own CEO

Five people enter the world. Each starts as their own boss with rank 0.

person	→ boss
Kyle	Kyle (CEO)
Eddie	Eddie (CEO)
Nambi	Nambi (CEO)
Arye	Arye (CEO)
Rock	Rock (CEO)

person	rank
Kyle	0
Eddie	0
Nambi	0
Arye	0
Rock	0

Kyle Eddie Nambi Arye Rock
(CEO) (CEO) (CEO) (CEO) (CEO)

STEP 1 Eddie hires Kyle `union('Kyle', 'Eddie')`

What happens inside union:

```
find('Kyle') → Kyle is own boss → CEO is Kyle, rank 0
find('Eddie') → Eddie is own boss → CEO is Eddie, rank 0
Equal ranks (0 == 0) → tie → Eddie absorbs Kyle, bump Eddie's rank.

parent['Kyle'] = 'Eddie'
rank['Eddie'] = 1
```

person	→ boss
Kyle	Eddie
Eddie	Eddie (CEO)
Nambi	Nambi (CEO)
Arye	Arye (CEO)
Rock	Rock (CEO)

person	rank
Kyle	0 (stale)
Eddie	1
Nambi	0
Arye	0
Rock	0

```
      Eddie (CEO, rank 1)      Nambi   Arye   Rock
      |
      Kyle
```

Why bump Eddie's rank? The new tree is height 1 (Kyle is one hop below Eddie). Rank tracks height. Two equal-rank trees merging is the only case where height grows. It grew by exactly 1.

STEP 2 Arye hires Nambi `union('Nambi', 'Arye')`

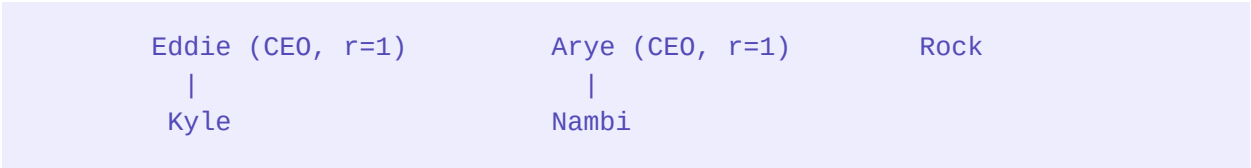
Same situation — both are their own CEOs with rank 0.

```
find('Nambi') → CEO Nambi, rank 0
find('Arye') → CEO Arye, rank 0
Equal ranks → Arye absorbs Nambi, bump Arye's rank.

parent['Nambi'] = 'Arye'
rank['Arye'] = 1
```

person	→ boss
Kyle	Eddie
Eddie	Eddie (CEO)
Nambi	Arye
Arye	Arye (CEO)
Rock	Rock (CEO)

person	rank
Kyle	0 (stale)
Eddie	1
Nambi	0 (stale)
Arye	1
Rock	0



STEP 3 Arye acquires Eddie's company `union('Eddie', 'Arye')`

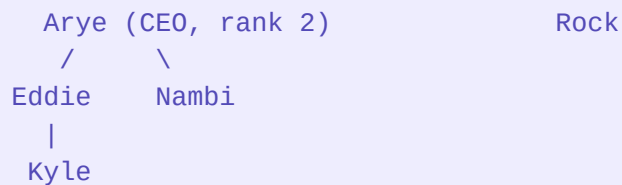
Both Eddie and Arye are CEOs of equal-sized companies (rank 1 each). Even though this is a 'big' merger, rank only goes up by 1 — never by 2.

```
find('Eddie') → CEO Eddie, rank 1
find('Arye') → CEO Arye, rank 1
Equal ranks → Arye absorbs Eddie, bump Arye's rank.

parent['Eddie'] = 'Arye'
rank['Arye'] = 2
```

person	→ boss
Kyle	Eddie
Eddie	Arye
Nambi	Arye
Arye	Arye (CEO)
Rock	Rock (CEO)

person	rank
Kyle	0 (stale)
Eddie	1 (stale)
Nambi	0 (stale)
Arye	2
Rock	0



Why rank 2 and not 4? Heights don't *add* when you stack two trees — only one new level appears (the new edge between the two roots). Both old trees were height 1; the combined tree is height 2. Rank goes from 1 → 2, never +2 from each side.

Subtle but important: we changed `parent['Eddie']`, not `parent['Kyle']`. We attach the **CEOs**, not the people we were originally given. Kyle still reports to Eddie. But Eddie now reports to Arye, so Kyle's full chain becomes Kyle → Eddie → Arye.

STEP 4 Rockwell acquires Arye's company `union('Arye', 'Rock')`

Imagine Rock has been busy hiring elsewhere and Rockwell is now a tall company with rank 3. Now Rockwell acquires Arye's company. Arye has rank 2; Rock has rank 3. **Different ranks.**

```
# Pretend Rock built up his company separately:
rank['Rock'] = 3

find('Arye') → CEO Arye, rank 2
find('Rock') → CEO Rock, rank 3
Different ranks → smaller goes under bigger → Arye absorbs into Rock.
NO rank bump because Rock's tree was already taller.

parent['Arye'] = 'Rock'
# rank['Rock'] stays at 3 – unchanged
```

person	→ boss
Kyle	Eddie
Eddie	Arye
Nambi	Arye
Arye	Rock
Rock	Rock (CEO)

person	rank
Kyle	0 (stale)
Eddie	1 (stale)
Nambi	0 (stale)
Arye	2 (stale)
Rock	3

```
Rock (CEO, rank 3)
|
Arye
/ \
Eddie Nambi
|
Kyle
```

The big lesson: a huge acquisition happened but rank didn't bump. Why? Rock's tree was already taller than Arye's. Arye's whole tree slid in as a 2-deep branch under Rock, but Rock already had 3-deep paths elsewhere. The deepest path didn't change. **This is rank doing its job — preventing the tree from getting unnecessarily tall.**

STEP 5 Someone asks: who does Kyle work for? `find('Kyle')`

Kyle's chain right now is Kyle → Eddie → Arye → Rock — three hops. This is where path compression kicks in. **find** does its job in two passes.

Pass 1 — walk up to discover the CEO

```
ceo = 'Kyle'
parent['Kyle'] = 'Eddie' (not self) → ceo = 'Eddie'
parent['Eddie'] = 'Arye' (not self) → ceo = 'Arye'
parent['Arye'] = 'Rock' (not self) → ceo = 'Rock'
parent['Rock'] = 'Rock' (self!) → STOP
```

CEO is Rock. Nothing rewritten yet.

Pass 2 — walk up again, rewriting along the way

```
person = 'Kyle'
parent['Kyle'] is 'Eddie', not 'Rock' → REWRITE
next_boss = 'Eddie'; parent['Kyle'] = 'Rock'; person = 'Eddie'

person = 'Eddie'
parent['Eddie'] is 'Arye', not 'Rock' → REWRITE
next_boss = 'Arye'; parent['Eddie'] = 'Rock'; person = 'Arye'

person = 'Arye'
parent['Arye'] is 'Rock' (the CEO) → STOP

Return 'Rock'.
```

person	→ boss
Kyle	Rock
Eddie	Rock
Nambi	Arye
Arye	Rock
Rock	Rock (CEO)

person	rank
Kyle	0 (stale)
Eddie	1 (stale)
Nambi	0 (stale)
Arye	2 (stale)
Rock	3

```
      Rock (CEO, rank 3)
     /  |  \
    Kyle Eddie Arye
       |
      Nambi ← unchanged
```


Critical insight: Kyle and Eddie now report directly to Rock. But **Nambi was not on this path** — we never walked through Nambi during `find('Kyle')`. So Nambi still points to Arye. Nambi's chain will only flatten when someone calls `find('Nambi')`. **Path compression is lazy: it only flattens the path that find actually walked.**

Note about rank values: `rank['Rock']` stays at 3, even though the tree is now actually height 1 from Kyle/Eddie/Arye and height 2 from Nambi. Rank is allowed to be a stale upper bound on height. It still works as a tiebreaker in future unions.

The full code

Here's the entire algorithm with both optimizations. Every line maps to something we just walked through.

```

# parent[person] = the person's direct boss
# If parent[person] == person, that person is a CEO (a root)
parent = {}

# rank[person] = roughly how tall this person's company is.
# Only meaningful when the person is a CEO. Stale otherwise.
rank = {}

def make_person(name):
    # A new person joins as their own boss (a company of one)
    parent[name] = name
    rank[name] = 0

def find(person):
    # Step 1: find the CEO (no rewriting yet)
    ceo = person
    while parent[ceo] != ceo:
        ceo = parent[ceo]

    # Step 2: walk up again, rewriting everyone to point at the CEO
    while parent[person] != ceo:
        next_boss = parent[person]    # remember who person USED to point to
        parent[person] = ceo          # rewrite: report directly to CEO
        person = next_boss            # move up the old chain

    return ceo

def union(a, b):
    ceo_a = find(a)
    ceo_b = find(b)

    if ceo_a == ceo_b:
        return                                # already same company

    # Smaller-ranked CEO gets absorbed under the bigger one
    if rank[ceo_a] < rank[ceo_b]:
        parent[ceo_a] = ceo_b                # no rank change
    elif rank[ceo_a] > rank[ceo_b]:
        parent[ceo_b] = ceo_a                # no rank change
    else:
        parent[ceo_b] = ceo_a                # equal: pick one
        rank[ceo_a] += 1                     # the only place rank ever grows

def connected(a, b):
    return find(a) == find(b)

```

Confusions resolved (the FAQ)

Where do parent pointers come from in the first place?

Only union creates them. Initially everyone points to themselves. The very first non-self pointer is always written by some union call. find never invents new pointers — it only reads existing ones (and optionally compresses them).

Is path compression a separate phase you run after building the tree?

No. It is a side effect of find. After all your unions, the tree may still be tall — that's fine. The algorithm doesn't sweep through and flatten everything. It just waits until someone calls find on a specific node, and then flattens the path that call walked. Nodes nobody asks about stay tall forever, and that costs nothing.

Who writes to the parent dictionary?

Both union and find. union writes exactly one entry to merge two roots. find writes many entries as it flattens the path it walked (path compression). union creates new structure; find flattens existing structure.

Does rank track number of nodes or height?

Height (roughly). What matters for find's speed is how many hops you walk, which is bounded by the tree's height. A 1000-node flat tree is fast; a 5-node stick is slow. Rank tracks height because that's what we want to control.

Why does rank only bump by 1, even for huge mergers?

Stacking two equal-height trees adds exactly one new level — the edge between the two roots. Heights don't add. Two height-1 trees merged make a height-2 tree, never height-4. Rank reflects that.

Why doesn't rank bump when ranks differ?

Sliding a smaller tree under a larger one doesn't make the larger tree taller. The deepest path was already in the larger tree, and it didn't change. So rank stays the same.

How do I know which company is bigger?

Store the size at the CEO. To compare two people's companies, walk each person up to their CEO via find, then compare the CEOs' rank values. Non-CEOs have stale rank values that we ignore.

Is rank read by find?

No. find only walks parent pointers. Rank is used only inside union, to pick the merge direction. If you deleted rank entirely, find would still give correct answers — just slower in worst-case tree shapes.

The two-minute recap

If you remember nothing else, remember this:

The data structure: one dictionary, `parent`. Maps each person to their boss. If you point to yourself, you're a CEO.

find(x): walk up the boss chain until you hit a CEO. Return the CEO. While walking, rewrite each visited node to point directly at the CEO (path compression).

union(a, b): find both CEOs. If they're different, point the smaller-ranked one at the bigger-ranked one. If they're equal, pick one and bump its rank by 1.

rank: a small helper dictionary. Stores roughly the tree's height at each CEO. Only used inside union to decide attach direction. Never read by find. Bumps only when two equal-rank CEOs merge — by exactly 1, never more.

Where you'll see this used

Kruskal's MST: sort edges by weight; for each edge (u,v) , if $\text{find}(u) \neq \text{find}(v)$, add it and union the endpoints.

Connected components: count distinct roots after processing all edges.

Cycle detection in undirected graphs: when adding edge (u,v) , if $\text{find}(u) == \text{find}(v)$, it's a cycle.

Dynamic connectivity: answer 'are u and v connected?' as edges stream in.

Built around your story: Kyle, Eddie, Nambi, Arye, Rock — and the acquisitions that made you remember it.